Project 4 Report

Option B (AI Code)

Vaughn Cox

## Overview

This project implements two AI systems:

1. **Wumpus World** – a knowledge-based agent that uses a knowledge base (KB) and inference rules to explore a grid while avoiding pits and the Wumpus.
2. **Tic-Tac-Toe** – an adversarial search agent using **Minimax** and **Alpha-Beta pruning** to play perfect games.

The code is split into separate modules:

- agent.py, kb.py, world.py, wumpus.py, test_layouts.py for Wumpus World.
- game.py, minimax.py, alphabeta.py, opponents.py, tictactoe.py, tictactoe_experiments.py for Tic-Tac-Toe.

All runs produce JSON output with required metrics (success/result, runtime, moves, nodes evaluated, trace).

## Wumpus World Representation

The environment is a grid (4×4, 5×5, 7×7, 8×8) with:

- One start square (0,0)
- Hidden pits (P)
- Exactly one Wumpus (W)
- Percepts: breeze (adjacent pit) and stench (adjacent Wumpus)

The WumpusWorld class generates percepts from the layout. The agent maintains a KnowledgeBase that tracks:

- safe – cells proved to have **no pit and no Wumpus**
- unsafe – cells proved to contain a pit or the Wumpus

- pit_free / wumpus_free – separate sets for each hazard
- known_wumpus – deduced location
- pit_candidates – possible pit locations for each breezy cell (used for intersection inference)

**Inference Rules Implemented**

| Rule | Implementation |
|---|---|
| **No breeze** → adjacent cells are pit-free | mark_pit_free(neighbor) |
| **No stench** → adjacent cells are wumpus-free | mark_wumpus_free(neighb |
| **Breeze** → record possible pit candidates; intersect across breezy cells; if only one candidate remains, mark it unsafe | pit_candidates with intersection |
| **Stench** → possible Wumpus candidates; if only one, mark as known_wumpus and mark all other cells wumpus-free | propagate_wumpus_free |
| **If all but one neighbor are safe, the last is unsafe** | safe-neighbor count infere |

The agent only moves into cells that are both pit-free and wumpus-free (self.kb.safe). If no safe unvisited cell is reachable, no guessing.

**Tic-Tac-Toe Implementation**

The game supports a 3×3 board, alternating turns, terminal detection (win/loss/draw), and utility values (+1 for X win, 0 for draw, –1 for O win).

**Minimax** is implemented with full tree search. **Alpha-Beta pruning** is added as an optional configuration. The agent always plays as **X** (maximizing player). Two opponents are used:

- RandomOpponent – chooses uniformly among legal moves.
- ScriptedOpponent – deterministic: prefers center, then corners, then edges.

The tictactoe_experiments.py script runs:

- 10 games Minimax vs Random
- 5 games Minimax vs Scripted
- 10 games Alpha-Beta vs Random
- 5 games Alpha-Beta vs Scripted

All results are saved to tictactoe_experiments.json.

**Testing Summary**

Both modules include correctness checks in tests.py using Python's unittest framework to validate both the Wumpus World and Tic-Tac-Toe modules. For Wumpus World, the tests verify that percept generation correctly detects breezes and stenches, that the knowledge base updates after each percept, that the agent never moves into a square already known to be unsafe, and that the inference mechanism correctly marks pits when two breezy cells share a single common neighbor. For Tic-Tac-Toe, the tests check legal move generation, terminal state detection (win, loss, draw), Minimax's ability to choose a winning or blocking move, and that Alpha-Beta pruning returns the same move as Minimax while evaluating no more nodes. All tests pass successfully, confirming the correctness of both agents.

**Experimental Results**

**Wumpus World (KB agent)**

All layouts were run with config="kb", max_steps=100. Success = True for every layout – the agent never died. The table summarises key metrics:

| Layout | Grid Size | Moves | Runtime (ms) | Success |
|---|---|---|---|---|
| 4x4_easy | 4 | 13 | 2 | ✓ |
| 4x4_medium | 4 | 3 | 0 | ✓ |
| 4x4_hard | 4 | 0 | 0 | ✓ |

| Layout | Grid Size | Moves | Runtime (ms) | Success |
|---|---|---|---|---|
| 5x5_easy | 5 | 23 | 5 | ✓ |
| 5x5_medium | 5 | 12 | 2 | ✓ |
| 5x5_hard | 5 | 0 | 0 | ✓ |
| 7x7_easy | 7 | 58 | 11 | ✓ |
| 7x7_medium | 7 | 3 | 0 | ✓ |
| 7x7_hard | 7 | 0 | 0 | ✓ |
| 8x8_easy | 8 | 73 | 11 | ✓ |
| 8x8_medium | 8 | 53 | 15 | ✓ |
| 8x8_hard | 8 | 0 | 0 | ✓ |

**Observations**

- Layouts where the start cell has **no breeze and no stench** (all "easy" and some "medium") allow the agent to move out and explore many safe cells.
- Layouts where the start cell has a **breeze** (most "hard" and some "medium") result in **0 moves** because no adjacent cell can be proved safe – the agent correctly stops.
- The agent never fell into a pit or was eaten by the Wumpus (success=True in all runs).
- Runtime remains low because inference is local and the grids are small.

**Tic-Tac-Toe (Minimax vs Alpha-Beta)**

The experiments ran 30 games in total. Both algorithms **won every game** (no draws or losses). The table below compares average performance:

| Configuration | Opponent | Games | Wins | Avg Nodes Evaluated | Avg Runtime (ms) |
|---|---|---|---|---|---|
| Minimax | Random | 10 | 10 | ~557,536 | ~1,336 |
| Minimax | Scripted | 5 | 5 | ~557,487 | ~1,347 |
| Alpha-Beta | Random | 10 | 10 | ~18,915 | ~48 |
| Alpha-Beta | Scripted | 5 | 5 | ~19,213 | ~47 |

**Observations**

- Alpha-Beta reduces node evaluations by **~97%** compared to plain Minimax.
- Runtime drops from ~1.3 seconds to under 50 milliseconds.
- Move quality is identical – both algorithms win every game, confirming that pruning does not affect the final decision.

**Analysis**

**How the Wumpus Agent Infers Safe Squares**

The agent uses negative percepts (no breeze, no stench) as strong evidence. For example, on 8x8_easy, at step 0 (start) it perceives no breeze and no stench, so it marks (1,0) and (0,1) as both pit-free and wumpus-free → safe. It then moves into those cells.

When a positive percept (breeze or stench) appears, the agent records candidates. For instance, on 8x8_medium at step 10, the agent at (5,1) perceives a breeze. It records possible pits among its neighbours that are not yet pit-free. Later, another breezy cell provides a second candidate set. The intersection of these sets narrows down the pit location, which is then marked unsafe.

Concrete inference example (from wumpus_8x8_medium.json trace):

- Step 10: at (5,1), breeze = True → possible pits = {(5,2), (4,1), (6,1)}.
- Step 14: at (1,1), breeze = True → possible pits = {(1,2), (0,1), (2,1)}.

- No overlap yet. Later, step 20 at (2,2) breeze = True → possible pits = {(2,3), (1,2), (3,2)}.
  Intersection of candidates from (1,1) and (2,2) is {(1,2)} → cell (1,2) becomes a known pit (unsafe). The agent then avoids it. This deduction is purely logical – the agent never "guesses".

**Why Minimax Chooses a Particular Move**

Consider a near-terminal board (from a scripted opponent game):

X X O

_ O _

X _ _

It is X's turn. Minimax evaluates every possible move. Moving to (1,0) creates a vertical line of three X's, winning immediately. The utility of that move is +1. Any other move (e.g., (1,2)) would not win immediately and could allow O to block or draw. Minimax selects the move with the highest utility under optimal opponent play – here the immediate win is the best. This matches the recorded trace where the AI plays (1,0) and wins on the next step.

**Alpha-Beta Pruning Comparison**

Alpha-Beta pruning reduces the search tree by cutting off branches that cannot influence the final decision. In Tic-Tac-Toe, the full Minimax tree has about 549,946 leaf nodes at the first move. Alpha-Beta evaluates only ~18,915 nodes on average – a reduction of **97%**. The runtime improves proportionally (from ~1.3 sec to ~0.05 sec). This demonstrates that pruning is highly effective even in a small game, and it preserves optimal play (same move choices as Minimax).

**Validation Evidence**

- **Wumpus World:**
  - Manually verified percept generation on all layouts.
  - Checked that the agent never steps into a cell marked unsafe (confirmed by JSON traces – no death entries).
  - Confirmed KB updates after each percept by printing kb.safe and kb.unsafe during debugging.

- **Tic-Tac-Toe:**
    - Unit tests for legal moves, terminal detection, and utility.
    - Ran Minimax on a board with one move to win – it picks the winning move.
    - Compared Minimax and Alpha-Beta on 30 games – same move choices every time, but Alpha-Beta always evaluates fewer nodes.

All JSON outputs include the required fields
(problem, instance, config, success/result, runtime_ms, moves_taken/nodes_evaluated, trace).

**AI Disclosure Appendix**

This project was completed under **Option B** (AI used for coding).

**AI Tools Used**

- ChatGPT (OpenAI) – primary assistant for code generation, debugging, and structuring.
- DeepSeek – used to organize experiment results into tables. Also used to help debug based on results in JSON files.

**What the Tools Were Used For**

- Setting up the initial Wumpus World environment, KB, and agent loop.
- Implementing pit candidate intersection and propagation logic.
- Optimizing and fixing Minimax and Alpha-Beta search functions.
- Generating the tictactoe_experiments.py script.
- Debugging inference errors (e.g., why the agent was moving into pits).
- Formatting JSON outputs and creating summary tables.
- Formatting and grammar work on report.

**Example Prompts**

1. *"Implement a Wumpus World knowledge base with separate pit_free and wumpus_free sets, and a propagation method for when the Wumpus location is known."*

2. *“My agent moves into a pit on 7x7_easy. Look at the trace and tell me why the pit at (2,2) is not marked unsafe.”*
3. *“Write a Tic-Tac-Toe experiment script that runs 10 games Minimax vs Random, and the same for Alpha-Beta. Save all results to JSON.”*

**Verification Steps**

- All generated code was reviewed and tested on small examples.
- Wumpus traces were manually inspected to ensure no unsafe moves.
- Tic-Tac-Toe games were run with both algorithms; move sequences were checked for optimality.
- The final code matches the project specification and produces correct JSON outputs.

**Conclusion**

The Wumpus World agent successfully explores safe cells using logical inference and stops conservatively when uncertainty remains, so it never dies. The Tic-Tac-Toe agent plays omptimally, and Alpha-Beta pruning dramatically reduces search cost without sacrificing move quality. All required runs were completed, and the outputs are fully documented in JSON files. The project demonstrates a clear separation of knowledge-based reasoning and adversarial search, meeting the objectives of CS 326 Project 4.